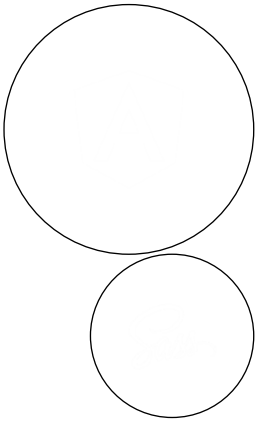
Desarrollo Web Integrado · 100000ST61

Fundamentos de Angular



Generación de componentes y estilos con SASS
Unidad 3 · Semana 11 · Sesión 11

Mag. Ing. Marco Antonio Flores Torres · UTP Lima Sur

1

RUTA DE LA SESIÓN

¿Cómo vamos a trabajar hoy?



Inicio

Rompe hielo, repaso y saberes previos

8 min



Utilidad

Repaso retador, logro e importancia

7 min



Transformación

Requisitos, Angular, componentes y SASS

30 min



Práctica

3 ejercicios: inicial, intermedio, avanzado

10 min



Cierre

Conclusiones, evaluación y mensaje final

5 min

Laboratorio dirigido 11: “TechStore Web” — catálogo de productos con componentes Angular y estilos SASS (60 a 90 min)

2

Inicio

Despertar el interés y activar lo que ya sabes

1

3

1 · INICIO · ROMPE HIELO

Rompe hielo: “Desarma la app”

1. Elige una app

En equipos de 4, abran en el celular una pantalla de Yape, Rappi, Spotify o Netflix.

2. Desármala (3 min)

En una hoja dibujen rectángulos y nombren cada pieza: barra superior, tarjeta, botón, lista, menú inferior…

3. Compartan

Dos equipos muestran su dibujo. Pregunta guía: ¿qué piezas se repiten en la misma pantalla?

Desarrollo Web Integrado · Sesión 11 · UTP

4

4

1 · INICIO · REPASO RÁPIDO

Repaso rápido de la sesión anterior

1. ¿Qué significa JWT y para qué lo usamos en nuestra API?
2. ¿Qué anotación de Spring hace que un método se ejecute dentro de una transacción?
3. ¿Qué método HTTP usamos para crear un recurso y qué código de estado devuelve normalmente?

5

1 · INICIO · IMPORTANCIA

¿Por qué te importa Angular hoy?

Hasta hoy: el back-end

Tu API Spring Boot ya guarda, consulta y protege datos con JPA, Spring Security y JWT. Pero el usuario final no usa Postman.

Desde hoy: el front-end

Angular construye la interfaz que el cliente ve y toca, y se conecta a tu API REST para mostrar sus datos.

Meta de la unidad 3: integrar tu back-end con un front-end mediante REST en el proyecto integrador

- **Empleabilidad:** los equipos de desarrollo buscan perfiles que entiendan front-end y back-end.
- **Tu proyecto final:** se evalúa en la semana 18 e integra ambas capas.
- **Versión vigente:** Angular 22 (junio de 2026), con TypeScript y componentes modernos.

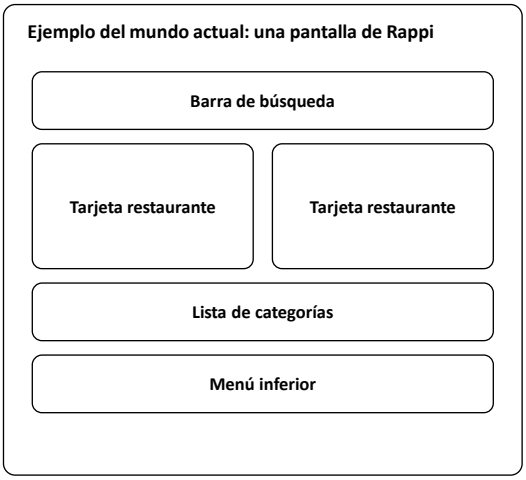
6

1 · INICIO · SABERES PREVIOS

¿Qué sabes ya? Saberes previos

Responde en voz alta o por el chat:

- ¿Qué función cumplen HTML, CSS y JavaScript en una página web?
- ¿Has visto una página que cambia sin recargar por completo? ¿Dónde?
- ¿Qué crees que es un “componente” en una app como Yape o Rappi?
- ¿Qué problema tiene mantener un CSS de 2 000 líneas en un solo archivo?



Desarrollo Web Integrado · Sesión 11 · UTP

7

7

Utilidad

Qué lograrás y por qué vale la pena

2

8

2 · UTILIDAD · REPASO RETADOR

Repaso retador: 3 preguntas de la sesión 10

1. Un cliente llama a un endpoint protegido con un JWT vencido. ¿Qué código HTTP debe devolver la API y por qué no es 403?

2. En un método @Transactional se guarda un Pedido y luego se lanza una RuntimeException al descontar el stock. ¿Qué pasa con el Pedido ya guardado?

3. Escribe la consulta JPQL que lista los productos con stock menor a 5, ordenados por precio de menor a mayor.

9

2 · UTILIDAD · RESPUESTAS

Respuestas y explicación breve

1. Respuesta: 401

401 Unauthorized: no se pudo autenticar la identidad (token vencido o inválido). 403 Forbidden se usa cuando el usuario sí está autenticado pero su rol no tiene permiso.

2. Respuesta: rollback

Spring revierte la transacción por defecto ante una RuntimeException: el Pedido guardado se deshace y la base de datos queda consistente.

3. Respuesta: JPQL

SELECT p FROM Producto p WHERE p.stock < 5 ORDER BY p.precio ASC. JPQL usa nombres de entidad y atributos, no de tabla ni de columna.

10

2 · UTILIDAD · LOGRO DE LA SESIÓN

Logro de aprendizaje de la sesión

Logro de la unidad 3 (sílabo)

Al finalizar la unidad, el estudiante integra proyectos de Back-end con proyectos de Front-end mediante métodos REST en un proyecto integrador.

Al finalizar la sesión, tú podrás:

- Crear un proyecto Angular con Angular CLI y explicar su estructura.
- Generar componentes y enlazar datos y eventos en sus plantillas.
- Aplicar estilos con SASS: variables, anidamiento, parciales y mixins.
- Aplicarlo en tu carrera: construir el catálogo de TechStore, la base de la interfaz de tu proyecto final.

11

2 · UTILIDAD · IMPORTANCIA

Utilidad e importancia de aprender Angular

Empleabilidad

Angular es un framework muy usado en aplicaciones empresariales. Dominar front-end y back-end te hace un perfil más completo.

Productividad

Un componente se escribe una vez y se reutiliza muchas veces: una tarjeta de producto sirve para 4 o para 400 productos.

Integración

Angular consume los endpoints REST de tu API Spring Boot. Es la pieza que conecta tu proyecto de punta a punta.

Mantenibilidad

SASS organiza los estilos con variables y parciales: cambias un color en un lugar y se actualiza toda la app.

12

2 · UTILIDAD · EJEMPLOS ACTUALES

Ejemplos prácticos del mundo actual

Catálogo de e-commerce

Tarjetas de producto, filtros y carrito, como la tienda que vamos a construir (TechStore Perú).

Panel de ventas

Tarjetas de indicadores (KPI), tablas y gráficos que se actualizan con datos de una API.

Banca y pagos en línea

Formularios validados, historial de movimientos y estados de operación con colores.

Intranet o campus virtual

Menús, rutas y vistas por rol: justo lo que combina con Spring Security y JWT.

13

Transformación

Requisitos, fundamentos de Angular, componentes y SASS

3

14

3 · TRANSFORMACIÓN · REQUISITOS

Antes de Angular: ¿qué necesitas saber?

HTML y CSS

Estructura y apariencia de una página; el DOM.

JavaScript ES6+

Módulos, funciones flecha, arrays y objetos.

TypeScript

JavaScript con tipos, clases e interfaces.

HTTP, REST y JSON

Cómo el front pide datos a tu API Spring Boot.

Node.js y npm

Entorno y gestor de paquetes para Angular CLI.

Idea clave: Angular no reemplaza lo que ya sabes; lo organiza en componentes reutilizables

Desarrollo Web Integrado · Sesión 11 · UTP

15

15

3 · TRANSFORMACIÓN · PRINCIPIO 1

Principio 1: cliente-servidor y SPA



Sitio tradicional (multipágina)

Cada clic pide al servidor una página HTML completa y el navegador la recarga entera. Es simple, pero se siente lento.

SPA: Single Page Application

Se descarga la aplicación una sola vez. Después solo viajan datos JSON y Angular actualiza únicamente la parte de la pantalla que cambió.

Desarrollo Web Integrado · Sesión 11 · UTP

16

16

3 · TRANSFORMACIÓN · PRINCIPIO 2

Principio 2: HTML, CSS y el DOM

HTML: estructura

```
<!-- Una tarjeta de producto -->
<article class="tarjeta">
  <h3>Laptop Lenovo</h3>
  <p class="precio">S/ 2499</p>
  <button>Comprar</button>
</article>
```

CSS: apariencia

```
/* Selector de clase + propiedades */
.tarjeta {
  border-radius: 12px;
  padding: 16px;
}
.tarjeta h3 { color: #dd0031; }
```

Lo esencial

- **DOM:** el navegador transforma el HTML en un árbol de nodos que JavaScript puede modificar.
- **Selectores CSS:** elementos (h3), clases (.tarjeta) e identificadores (#menu).
- **Problema:** modificar el DOM a mano se vuelve inmanejable cuando la pantalla crece.
- **Solución:** Angular describe la pantalla con componentes y actualiza el DOM por ti.

Desarrollo Web Integrado · Sesión 11 · UTP

17

17

3 · TRANSFORMACIÓN · PRINCIPIO 3

Principio 3: JavaScript moderno (ES6+)

JavaScript ES6+

```
// 1. Módulos: compartir código entre archivos
import { calcularIGV } from './impuestos';
export const IGV = 0.18;

// 2. const/let y plantillas de texto
const precio = 100;
console.log(`Total: S/ ${precio * (1 + IGV)}`);

// 3. Funciones flecha + filter/map
const nombres = productos
  .filter(p => p.stock > 0)
  .map(p => p.nombre);

// 4. Desestructuración y spread
const { nombre } = producto;
const copia = { ...producto, stock: 0 };
```

- **import / export:** dividen la app en archivos; Angular los usa en cada componente.
- **const y let:** sin var; const por defecto.
- **Flecha (=>):** funciones cortas, muy usadas con arrays.
- **Spread (...):** copia objetos sin mutar el original.

Desarrollo Web Integrado · Sesión 11 · UTP

18

18

3 · TRANSFORMACIÓN · PRINCIPIO 4

Principio 4: TypeScript = JavaScript con tipos

TypeScript

```
// Tipos básicos
let nombre: string = 'Ana';
const stock: number = 12;
let tags: string[] = ['laptop', 'oferta'];

// Interfaz: contrato de datos (como un DTO)
interface Producto {
  id: number;
  nombre: string;
  precio: number;
  stock?: number;   // ? = opcional
}

// Clase con modificadores de acceso
class Carrito {
  private items: Producto[] = [];
  agregar(p: Producto): void { this.items.push(p); }
}
```

- **Tipos:** el editor detecta errores antes de ejecutar (precio: 'abc' no compila).
- **Interfaces:** describen la forma del JSON que devuelve tu API.
- **Clases y decoradores:** Angular usa @Component, parecido a las anotaciones de Spring.

19

3 · TRANSFORMACIÓN · PRINCIPIO 5

Si sabes Spring Boot, ya conoces Angular

Spring Boot (lo que ya sabes)	Angular (lo que aprenderás)
Spring Initializr	Angular CLI: ng new
pom.xml con Maven	package.json con npm
mvn spring-boot:run	ng serve
@RestController / @Controller	@Component
@Service e inyección de dependencias	Servicios e inject() (sesión 14)
application.properties	angular.json y archivos de entorno
Entidad o DTO en Java	interface en TypeScript
Puerto 8080	Puerto 4200 en desarrollo

20

3 · TRANSFORMACIÓN · HERRAMIENTAS

Principio 6: Node.js, npm y Angular CLI

Node.js

Ejecuta JavaScript fuera del navegador; el CLI lo necesita. Usa una versión LTS reciente (Node 22 o superior).

npm

Gestor de paquetes: descarga las dependencias que declara package.json, como Maven con pom.xml.

Angular CLI (ng)

Crea proyectos, genera componentes, compila y levanta el servidor.

Verifica tu entorno (terminal)

```
# 1. Versión de Node y npm
node -v
npm -v

# 2. Instalar Angular CLI (una vez)
npm install -g @angular/cli

# 3. Comprobar
ng version

# Si falla en Windows PowerShell:
# abre una terminal nueva o usa
# cmd / Git Bash
```


21

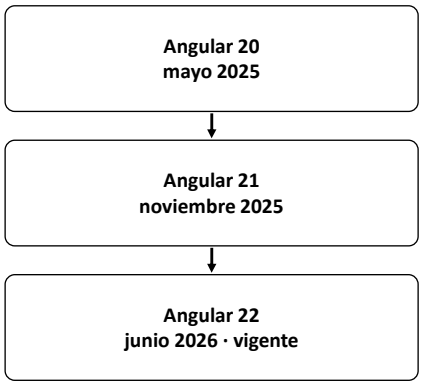
3 · TRANSFORMACIÓN · FUNDAMENTOS

¿Qué es Angular?

Framework de Google para construir aplicaciones web de una sola página (SPA) con TypeScript y componentes.

- **Basado en componentes:** la interfaz se arma con piezas reutilizables.
- **TypeScript de serie:** tipado y mejores herramientas de edición.
- **Todo incluido:** rutas, formularios, HTTP y pruebas.
- **Angular CLI:** genera código y proyectos con comandos.
- **Signals:** reactividad simple para actualizar la pantalla.

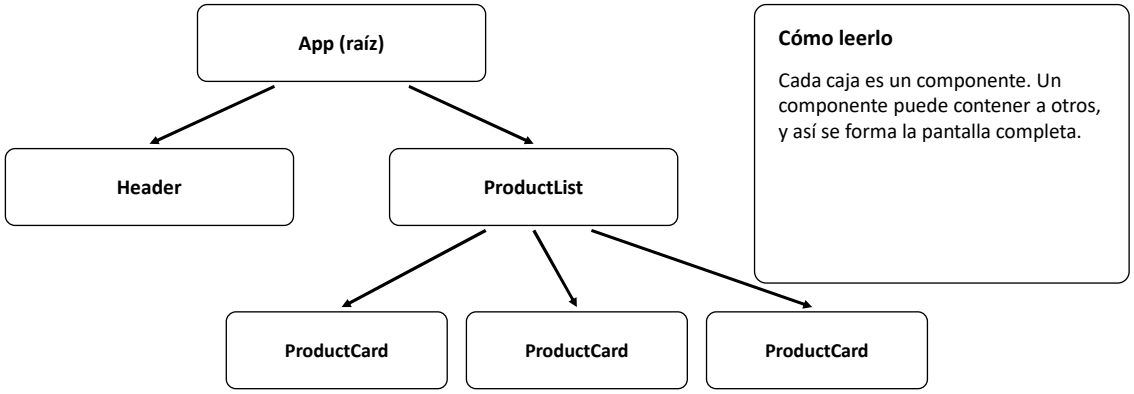
Versiones recientes



22

3 · TRANSFORMACIÓN · FUNDAMENTOS

Arquitectura: un árbol de componentes



Cómo leerlo

Cada caja es un componente. Un componente puede contener a otros, y así se forma la pantalla completa.

El mismo ProductCard se reutiliza una vez por producto.

3 · TRANSFORMACIÓN · FUNDAMENTOS

Crear el proyecto con Angular CLI

Terminal

```
# 1. Instalar Angular CLI (una vez)
npm install -g @angular/cli

# 2. Crear el proyecto con SCSS
ng new techstore-web --style=scss --ssr=false

# 3. Entrar al proyecto y ejecutarlo
cd techstore-web
ng serve

# 4. Abrir en el navegador
# http://localhost:4200

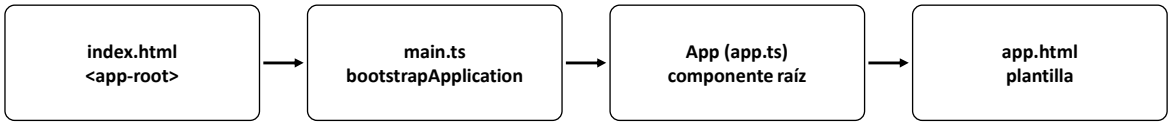
# Si pregunta por herramientas de IA,
# elige la opción sin herramientas.
```

Estructura del proyecto

```
techstore-web/
├─ angular.json      configuración
├─ package.json      dependencias
├─ public/           imágenes, favicon
└─ src/
   ├─ index.html     <app-root>
   ├─ main.ts        arranque
   ├─ styles.scss    estilos globales
   └─ app/
      ├─ app.ts      componente raíz
      ├─ app.html    su plantilla
      ├─ app.scss    sus estilos
      ├─ app.config.ts
      └─ app.routes.ts
```

3 · TRANSFORMACIÓN · FUNDAMENTOS

Así arranca tu aplicación



src/main.ts

```
import { bootstrapApplication }
  from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

- bootstrapApplication monta el componente raíz.
- appConfig guarda la configuración global (rutas, HTTP).
- Todo lo demás se compone dentro de App.

25

3 · TRANSFORMACIÓN · COMPONENTES

¿Qué es un componente?

Clase TypeScript

Datos y lógica: propiedades, métodos y señales. Archivo product-card.ts.

+

Plantilla HTML

Lo que se ve: HTML con enlaces a los datos de la clase. Archivo product-card.html.

+

Estilos SCSS

Apariencia solo de este componente. Archivo product-card.scss.

Componente = clase + plantilla + estilos, usado con su propia etiqueta: <app-product-card />

26

3 · TRANSFORMACIÓN · COMPONENTES

Anatomía de un componente

product-card.ts

```
import { Component, input } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { Producto } from '../../models/producto.model';

@Component({
  selector: 'app-product-card',
  imports: [CurrencyPipe],
  templateUrl: './product-card.html',
  styleUrl: './product-card.scss'
})
export class ProductCard {
  producto = input.required<Producto>();
}
```

- **selector:** la etiqueta HTML propia del componente.
- **imports:** componentes y pipes que usa su plantilla.
- **templateUrl:** archivo HTML de la plantilla.
- **styleUrl:** archivo SCSS exclusivo del componente.
- **class:** la lógica y los datos.

3 · TRANSFORMACIÓN · COMPONENTES

Generar componentes con Angular CLI

Terminal

```
# Forma larga
ng generate component components/product-card

# Forma corta (g = generate, c = component)
ng g c components/header
```

Archivos generados

```
components/product-card/
├─ product-card.ts        clase
├─ product-card.html      plantilla
├─ product-card.scss      estilos
└─ product-card.spec.ts   pruebas
```

- El CLI crea la carpeta y los cuatro archivos por ti.
- El selector queda como app-product-card.
- La clase se llama ProductCard.
- Importante: no se crean a mano; así se evitan errores de nombres y rutas.

3 · TRANSFORMACIÓN · COMPONENTES

Usar un componente dentro de otro

product-list.ts

```
import { ProductCard }
  from '../product-card/product-card';

@Component({
  selector: 'app-product-list',
  imports: [ProductCard],  // 1) importar
  templateUrl: './product-list.html'
})
export class ProductList {}
```

product-list.html

```
<!-- 2) usar el selector -->
<app-product-card />
<app-product-card />
<app-product-card />
```

Error frecuente

"'app-product-card' is not a known element": falta agregar ProductCard en el arreglo imports del componente que lo usa.

3 · TRANSFORMACIÓN · PLANTILLAS

Enlace de datos (1): texto y propiedades

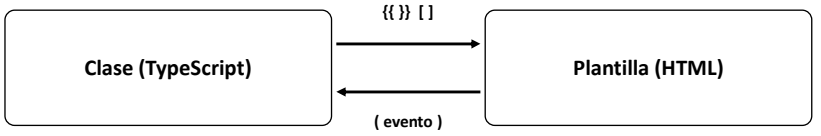
Tipo	Sintaxis	Ejemplo
Interpolación	{{ expresión }}	{{ producto().nombre }}
Propiedad	[propiedad]	<button [disabled]="agotado()">
Clase condicional	[class.nombre]	<div [class.card--agotado]="agotado()">
Pipe (formato)	valor | pipe	{{ precio | currency:'PEN':'S/ ' }}

Regla práctica

{{ }} muestra texto, [ ] envía datos a una propiedad o clase, y ( ) escucha eventos del usuario. Los datos fluyen de la clase hacia la plantilla.

3 · TRANSFORMACIÓN · PLANTILLAS

Enlace de datos (2): eventos



Dos direcciones

Datos hacia la vista y eventos hacia la clase.

contador.ts

```
export class Contador {
  visitas = signal(0);

  sumar() {
    this.visitas.update(v => v + 1);
  }
}
```

contador.html

```
<h2>Visitas: {{ visitas() }}</h2>

<!-- (evento)="método()" -->
<button (click)="sumar()">
  +1
</button>
```


31

3 · TRANSFORMACIÓN · PLANTILLAS

Signals: estado que se actualiza solo

Signals en un componente

```
import { signal, computed } from '@angular/core';

// Crear
cantidad = signal(1);
precio = signal(49.9);

// Leer: se llama como función
// cantidad()  ->  1

// Cambiar
this.cantidad.set(5);
this.cantidad.update(n => n + 1);

// Derivar: se recalcula solo
total = computed(
  () => this.cantidad() * this.precio()
);
```

- **signal():** guarda un valor reactivo.
- **set() / update():** cambian el valor.
- **computed():** valor calculado a partir de otros.
- **Resultado:** Angular actualiza solo lo que cambió en pantalla.

32

3 · TRANSFORMACIÓN · PLANTILLAS

Control de flujo: @if, @for y @switch

Plantilla HTML

```
@if (stock() > 0) {
  <span>Disponible</span>
} @else {
  <span>Agotado</span>
}

@for (p of productos(); track p.id) {
  <app-product-card [producto]="p" />
} @empty {
  <p>Sin resultados</p>
}

@switch (estado()) {
  @case ('ENVIADO') { <b>En camino</b> }
  @default { <b>Pendiente</b> }
}
```

- **@if / @else:** muestra bloques según una condición.
- **@for:** repite un bloque por cada elemento de la lista.
- **track:** identifica cada elemento; usa el id.
- **@empty:** se muestra cuando la lista está vacía.
- **@switch:** elige entre varios casos.

Desarrollo Web Integrado · Sesión 11 · UTP 33

33

3 · TRANSFORMACIÓN · PLANTILLAS

Entradas de un componente: input()

Padre: ProductList → **Hijo: ProductCard**

product-list.html

```
@for (p of productos(); track p.id) {
  <app-product-card [producto]="p" />
}
```

product-card.ts

```
producto = input.required<Producto>();

// En la plantilla se lee como signal:
// {{ producto().nombre }}
```

Recuerda

[producto]="p" en el padre envía el dato; input() en el hijo lo recibe. Para responder al padre se usará output() en la sesión 12.

Desarrollo Web Integrado · Sesión 11 · UTP 34

34

3 · TRANSFORMACIÓN · ACTUALIDAD

Novedades de Angular 22 que debes conocer

OnPush por defecto

Los componentes nuevos solo se revisan cuando cambian sus entradas o signals: más rendimiento.

Requisitos más nuevos

Requiere TypeScript 6 y una versión reciente de Node; Node 20 ya no es compatible.

Signal Forms estables

Formularios basados en signals, listos para producción. Los veremos en la sesión 13.

Servicios más simples

Nuevo decorador @Service() para servicios globales sencillos (sesión 14).

Herramientas con IA

Angular incorpora soporte para asistentes de IA mediante MCP y habilidades para agentes.

Versiones anuales

Las versiones mayores ahora salen cada 12 meses; la siguiente se espera en junio de 2027.

3 · TRANSFORMACIÓN · SASS

¿Por qué SASS?

CSS puro: repetitivo

```
.card { border: 1px solid #dd0031; }
.card h3 { color: #dd0031; }
.card button {
  background: #dd0031;
}
/* el color se repite 3 veces */
```

SCSS: variables y anidamiento

```
$primario: #dd0031;

.card {
  border: 1px solid $primario;
  h3 { color: $primario; }
  button { background: $primario; }
}
```

Escribes .scss → **Angular CLI compila con Sass** → **El navegador recibe CSS**

3 · TRANSFORMACIÓN · SASS

SASS dentro de Angular

Estilos globales: src/styles.scss

Se aplican a toda la aplicación: fuentes, color de fondo, reinicio de márgenes. Aquí se cargan las variables.

Estilos del componente: .scss

Solo afectan a la plantilla de ese componente (encapsulación). Un .card aquí no pisa a otro .card.

Configuración

```
# Al crear el proyecto:  ng new techstore-web --style=scss
# En cada componente:     styleUrl: './product-card.scss'
# (el CLI genera el archivo .scss automáticamente)
```

Desarrollo Web Integrado · Sesión 11 · UTP

37

37

3 · TRANSFORMACIÓN · SASS

Variables y anidamiento

SCSS (lo que escribes)

```
$color-primario: #dd0031;
$espacio: 16px;

.card {
  padding: $espacio;
  border: 1px solid $color-primario;

  h3 { margin: 0; }
  &:hover { background: #fdeef1; }
  &__precio { font-weight: 700; }
}
```

CSS (lo que recibe el navegador)

```
.card {
  padding: 16px;
  border: 1px solid #dd0031;
}
.card h3 {
  margin: 0;
}
.card:hover {
  background: #fdeef1;
}
.card__precio {
  font-weight: 700;
}
```

Desarrollo Web Integrado · Sesión 11 · UTP

38

38

3 · TRANSFORMACIÓN · SASS

Parciales y @use: organiza tus estilos

Carpeta

```
src/
├ scss/
│  ├ _variables.scss
│  └ _mixins.scss
├ styles.scss
└ app/
   └ components/
      └ product-card/
         └ product-card.scss
```

Declarar y usar variables

```
// scss/_variables.scss
$color-primario: #dd0031;
$radio: 12px;

// product-card.scss (cualquier componente)
@use '../../../scss/variables' as v;

.card { border-radius: v.$radio; }
```

Reglas

El parcial empieza con _ y se carga sin él. Se usa con el prefijo del módulo (v.$radio). No uses @import.

39

3 · TRANSFORMACIÓN · SASS

Mixins: estilos reutilizables

scss/_mixins.scss

```
@use 'variables' as v;

@mixin tarjeta($padding: 16px) {
  background: #fff;
  border-radius: v.$radio;
  padding: $padding;
}

@mixin desde($ancho) {
  @media (min-width: $ancho) {
    @content;
  }
}
```

Uso en un componente

```
@use '../../../scss/mixins' as m;

.card { @include m.tarjeta(24px); }

.menu {
  @include m.desde(768px) {
    display: flex;
  }
}
```

Beneficio

Escribes el estilo una vez y lo reutilizas con include en cualquier componente.

40

3 · TRANSFORMACIÓN · SASS

Más herramientas: @each y funciones

Mapas, bucles y funciones

```
@use 'sass:color';
@use 'sass:math';

$estados: ('ok': #1f8a70, 'alerta': #f2a900,
           'error': #dd0031);

@each $nombre, $c in $estados {
  .badge--#{$nombre} { background: $c; }
}

.btn:hover {
  background: color.adjust(#dd0031,
    $lightness: -8%);
}

@function rem($px) {
  @return math.div($px, 16) * 1rem;
}
```

- **Mapa:** pares clave: valor para agrupar datos.
- **@each:** genera una clase por cada estado.
- **color.adjust:** oscurece o aclara un color.
- **@function:** calcula valores reutilizables.

41

3 · TRANSFORMACIÓN · EJEMPLO 1

Ejemplo 1: componente ProductCard

product-card.ts

```
export interface Producto {
  id: number; nombre: string;
  categoria: string;
  precio: number; stock: number;
}

export class ProductCard {
  producto = input.required<Producto>();
  agotado = computed(
    () => this.producto().stock === 0
  );
  agregados = signal(0);

  agregar() {
    this.agregados.update(n => n + 1);
  }
}
```

product-card.html

```
<article class="card"
  [class.card--agotado]="agotado()">
  <span>{{ producto().categoria }}</span>
  <h3>{{ producto().nombre }}</h3>
  <p>{{ producto().precio |
      currency:'PEN':'S/ ' }}</p>
  @if (agotado()) {
    <b>Agotado</b>
  } @else {
    <b>Stock: {{ producto().stock }}</b>
  }
  <button [disabled]="agotado()"
    (click)="agregar()">Agregar</button>
</article>
```


42

3 · TRANSFORMACIÓN · EJEMPLO 2

Ejemplo 2: lista con @for, @empty y computed

product-list.ts

```
export class ProductList {
  productos = signal<Producto[]>([
    { id: 1, nombre: 'Laptop Lenovo',
      categoria: 'Laptops',
      precio: 2499, stock: 8 },
    { id: 2, nombre: 'Teclado Redragon',
      categoria: 'Accesorios',
      precio: 159, stock: 0 },
  ]);

  total = computed(
    () => this.productos().length);
  sinStock = computed(() =>
    this.productos()
      .filter(p => p.stock === 0).length);
}
```

product-list.html

```
<h2>Catálogo ({{ total() }})</h2>
<p>Sin stock: {{ sinStock() }}</p>

<div class="lista__grid">
  @for (p of productos(); track p.id) {
    <app-product-card [producto]="p" />
  } @empty {
    <p>No hay productos.</p>
  }
</div>
```

Desarrollo Web Integrado · Sesión 11 · UTP 43

43

3 · TRANSFORMACIÓN · EJEMPLO 3

Ejemplo 3: KpiCard con mixin SASS

kpi-card.ts

```
export class KpiCard {
  titulo = input.required<string>();
  valor = input.required<number>();
  variacion = input(0);
  sube = computed(() => this.variacion() >= 0);
}
```

kpi-card.html

```
<div class="kpi">
  <span>{{ titulo() }}</span>
  <strong>{{ valor() | number }}</strong>
  <span class="kpi__var"
    [class.kpi__var--baja]="!sube()">
    {{ sube() ? '▲' : '▼' }} {{ variacion() }}%
  </span>
</div>
```

kpi-card.scss

```
@use '../../../scss/mixins' as m;
@use '../../../scss/variables' as v;

.kpi {
  @include m.tarjeta(20px);
  &__var {
    color: v.$color-exito;
    &--baja { color: v.$color-primario; }
  }
}
```

Uso en el componente padre

```
<app-kpi-card
  titulo="Ventas del mes"
  [valor]="48250"
  [variacion]="12.5" />
```

Desarrollo Web Integrado · Sesión 11 · UTP 44

44

Práctica

4

Tres ejercicios retadores: inicial, intermedio y avanzado

45

4 · PRÁCTICA · EJERCICIO INICIAL

Nivel inicial

Ejercicio 1: contador de visitas

Escenario

TechStore quiere mostrar cuántas veces se vio un producto y destacar los más populares.

Qué debes hacer

- Genera el componente contador con Angular CLI.
- Crea un signal visitas que empiece en 0.
- Muestra el valor con interpolación y agrega los botones +1 y Reiniciar.
- Con @if, muestra “¡Producto popular!” cuando visitas() sea 10 o más.
- Pinta el mensaje con una variable SASS.

Instrucciones que usarás

- ng generate component
- signal, set y update
- Interpolación {{ }}
- Evento (click)
- @if
- Variable SCSS con @use

Desarrollo Web Integrado · Sesión 11 · UTP

46

46

4 · PRÁCTICA · EJERCICIO INTERMEDIO

Nivel intermedio

Ejercicio 2: estado de pedidos

Escenario

El área de logística necesita ver sus pedidos con una etiqueta de color según su estado, igual que el enum EstadoPedido de la entidad JPA.

Qué debes hacer

- Define el tipo EstadoPedido y la interface Pedido (espejo del back-end).
- Crea el componente pedido-lista con un signal de 3 pedidos.
- Recórrelos con @for (track p.id) y muestra @empty si no hay.
- Usa @switch para mostrar la etiqueta según el estado.
- Crea el mixin badge en SASS y aplícalo con un color por estado.

Instrucciones que usarás

- type e interface (TypeScript)
- signal<Pedido[]>
- @for, @empty, @switch
- Pipe currency
- Mixin con @include
- Parciales con @use

4 · PRÁCTICA · EJERCICIO AVANZADO

Nivel avanzado

Ejercicio 3: panel de inventario con filtro

Escenario

El jefe de almacén quiere filtrar el inventario por categoría y ver el valor total en stock y cuántos productos tienen stock bajo (menos de 5, como la consulta JPQL del repaso).

Qué debes hacer

- Usa la interface Producto, espejo de tu entidad Spring Boot.
- Crea inventario-panel (padre) y product-row (hijo con input.required).
- Con signal filtro y computed obtén: visibles, valorTotal y stockBajo.
- Dibuja un botón por categoría con @for y [class.activo].
- Con SASS, usa un mapa y @each para dar un color a cada categoría.

Instrucciones que usarás

- input.required y [producto]
- signal + computed
- @for con track
- [class.x] condicional
- Mapa y @each en SASS
- Interface = entidad Java

4 · PRÁCTICA · SOLUCIÓN 1

Solución 1: contador de visitas

contador.ts

```
export class Contador {
  visitas = signal(0);

  sumar()     { this.visitas.update(v => v + 1); }
  reiniciar() { this.visitas.set(0); }
}
```

contador.html

```
<h2>Visitas: {{ visitas() }}</h2>
@if (visitas() >= 10) {
  <p class="popular">¡Producto popular!</p>
}
<button (click)="sumar()">+1</button>
<button (click)="reiniciar()">Reiniciar</button>
```

Cómo se resuelve

1) signal(0) guarda las visitas y se lee como visitas().
2) update calcula desde el valor anterior; set lo reemplaza.
3) (click) enlaza cada botón con un método de la clase.
4) @if muestra el mensaje solo cuando visitas() >= 10.

contador.scss

```
@use '../../../scss/variables'
  as v;

.popular {
  color: v.$color-exito;
}
```


4 · PRÁCTICA · SOLUCIÓN 2

Solución 2: pedidos (clase y plantilla)

pedido-lista.ts

```
export type EstadoPedido =
  'PENDIENTE' | 'ENVIADO' | 'ENTREGADO';

export interface Pedido {
  id: number; cliente: string;
  total: number; estado: EstadoPedido;
}

@Component({
  selector: 'app-pedido-lista',
  imports: [CurrencyPipe], ...
})
export class PedidoLista {
  pedidos = signal<Pedido[]>([
    { id: 101, cliente: 'Lucía Ramos',
      total: 349.9, estado: 'PENDIENTE' },
    { id: 102, cliente: 'Carlos Pérez',
      total: 89.5, estado: 'ENVIADO' },
  ]);
}
```

pedido-lista.html

```
@for (p of pedidos(); track p.id) {
  <div class="fila">
    <span>#{{ p.id }} {{ p.cliente }}</span>
    <strong>{{ p.total | currency:'PEN':'S/ ' }}</strong>
    @switch (p.estado) {
      @case ('PENDIENTE') {
        <span class="badge--pend">Pendiente</span>
      }
      @case ('ENVIADO') {
        <span class="badge--env">Enviado</span>
      }
      @default {
        <span class="badge--ok">Entregado</span>
      }
    }
  </div>
} @empty { <p>No hay pedidos.</p> }
```

4 · PRÁCTICA · SOLUCIÓN 2

Solución 2: pedidos (estilos SASS)

scss/_mixins.scss (agrega)

```
@mixin badge($fondo, $texto: #fff) {
  display: inline-block;
  padding: 2px 10px;
  border-radius: 999px;
  background: $fondo;
  color: $texto;
}
```

Idea clave

Un mixin con parámetros evita repetir el mismo bloque de estilos en cada estado.

pedido-lista.scss

```
@use '../../../scss/variables' as v;
@use '../../../scss/mixins' as m;

.fila {
  @include m.tarjeta(12px);
  display: flex;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 8px;
}

.badge {
  &--pend {
    @include m.badge(v.$color-alerta,
                     v.$color-secundario);
  }
  &--env { @include m.badge(#5b4b8a); }
  &--ok  { @include m.badge(v.$color-exito); }
}
```


4 · PRÁCTICA · SOLUCIÓN 3

Solución 3: inventario (padre y plantilla)

inventario-panel.ts

```
export class InventarioPanel {
  productos = signal<Producto[]>(DATOS);
  filtro = signal('Todas');

  categorias = computed(() => ['Todas',
    ...new Set(this.productos().map(p => p.categoria))]);

  visibles = computed(() => {
    const f = this.filtro();
    return f === 'Todas' ? this.productos()
      : this.productos().filter(p => p.categoria === f);
  });
  valorTotal = computed(() => this.visibles()
    .reduce((s, p) => s + p.precio * p.stock, 0));
  stockBajo = computed(() => this.visibles()
    .filter(p => p.stock < 5).length);

  filtrar(c: string) { this.filtro.set(c); }
}
```

inventario-panel.html

```
<h2>Inventario TechStore</h2>

<div class="filtros">
  @for (c of categorias(); track c) {
    <button [class.activo]="c === filtro()"
      (click)="filtrar(c)">{{ c }}</button>
  }
</div>

<p>Valor: {{ valorTotal() | currency:'PEN':'S/ ' }}</p>
<p>Stock bajo: {{ stockBajo() }}</p>

@for (p of visibles(); track p.id) {
  <app-product-row [producto]="p" />
}
```

4 · PRÁCTICA · SOLUCIÓN 3

Solución 3: inventario (hijo y SASS)

product-row.ts (clase)

```
export class ProductRow {
  producto = input.required<Producto>();
  bajo = computed(() => this.producto().stock < 5);
  cat = computed(
    () => this.producto().categoria.toLowerCase());
}
```

product-row.html

```
<div class="fila" [class.fila--bajo]="bajo()">
  <span [class]="'tag tag-' + cat()">
    {{ producto().categoria }}
  </span>
  <span>{{ producto().nombre }}</span>
  <strong>Stock: {{ producto().stock }}</strong>
</div>
```

product-row.scss

```
$categorias: (
  'laptops': #dd0031,
  'accesorios': #5b4b8a,
  'monitores': #1f8a70
);

@each $nombre, $color in $categorias {
  .tag-#{$nombre} { background: $color; }
}
.tag { color: #fff; padding: 2px 10px; }
.fila--bajo { background: #fdeef1; }
```

Idea clave

El mapa y @each generan una clase por categoría sin repetir código.

Desarrollo Web Integrado · Sesión 11 · UTP

53

53

Cierre

Construimos las conclusiones juntos y medimos lo aprendido

5

54

5 · CIERRE · DINÁMICA

Dinámica: “3-2-1” para construir conclusiones

3 ideas clave	2 errores a evitar	1 duda
Qué entendí del tema que me tocó.	Qué fallaría si lo hago mal.	Qué aún quiero aclarar.

Temas que se reparten entre los equipos

Componente	**Plantilla y binding**	**Signals**
@if, @for, @switch	**input()**	**SASS**

55

5 · CIERRE · EVALUACIÓN · PREGUNTA 1

Evaluación: pregunta 1 de 3

Necesitas crear un componente llamado product-card dentro de src/app/components. ¿Qué comando usas?

A ng new components/product-card	**B** ng generate component components/product-card
C npm create product-card	**D** ng serve components/product-card

56

5 · CIERRE · EVALUACIÓN · PREGUNTA 2

Evaluación: pregunta 2 de 3

En <app-product-card [producto]="p" />, ¿qué representa [producto]?

- **A** Una interpolación que muestra texto en pantalla
- **B** Un evento que escucha los clics del usuario
- **C** Un property binding que envía el dato p al input del componente hijo
- **D** Una regla de estilos SCSS

Desarrollo Web Integrado · Sesión 11 · UTP 57

57

5 · CIERRE · EVALUACIÓN · PREGUNTA 3

Evaluación: pregunta 3 de 3

¿Qué afirmación sobre SASS en Angular es correcta?

- **A** El navegador lee los archivos .scss directamente
- **B** Con @use se cargan variables y mixins de parciales y el CLI compila todo a CSS
- **C** Las variables se declaran con -- y nunca se compilan
- **D** Los mixins se definen con @function y se llaman con @use

Desarrollo Web Integrado · Sesión 11 · UTP 58

58

Sesión 11 completada

¡Sigue construyendo!

Gracias por tu participación. Practica lo aprendido, completa el laboratorio y comparte tus dudas y conclusiones en el foro del curso.

Tu app de hoy será la interfaz de tu proyecto final.

Practica hoy: termina el laboratorio y prueba el reto adicional.

Participa en el foro del curso: comparte tu conclusión y tus dudas.

Próxima sesión: rutas de Angular y comunicación entre componentes.

59